A Reconfigurable SensorNet for Industry 5.0 Applications using Arduino Due/ESP32 and Xbee S2C Based on IEEE 802.15.4 Protocol with Programmable Sensor Array

H. R. Jayetileke[1], W. R. de Mel[2]
Dept. of Materials and Mechanical Technology
University of Sri Jayewardenepura
Homagama, Sri Lanka.
hrjayetileke@sjp.ac.lk[1]
wrdemel@sjp.ac.lk[2]

S. C. Mukhopadhyay
School of Engineering (44WR 134)
Macquarie University
NSW 2109 Australia.
Subhas.Mukhopadhyay@mq.edu.au

***Abstract*— This paper presents a wireless sensing platform for Industrial applications such as Industrial 5.0. It uses Xbee S2C and Arduino Due/ESP32 for wireless communication and offers customizable data resolution, sensor array size and synchronization at the same sampling iteration. The system effectively monitors reference trajectories up to 20 Hz and eliminates costly wired connections, improving data acquisition efficiency. This study contributes to the development of wireless telemetry systems for even Industrial 5.0 implementation.**

Keywords—Xbee S2C; IEEE 802.15.4 data protocol; IoT; IIoT; Arduino Due; ESP32; SensorNet

I. Introduction

Smart manufacturing, commonly referred to as Industry 5.0, enhances output by increasing productivity, efficiency, and safety [1][2]. Artificial Intelligence (AI) and Machine Learning (ML) powered high-precision automated machinery minimizes human effort in physical and cognitive tasks [3][4]. For data collecting and decision-making, networked sensors are used [3]. Wireless technology connects devices like sensors and actuators while maintaining Quality of Service (QoS) [5][6]. To fulfill QoS criteria, communication delays and package losses must be kept to a minimum. Industrial Internet of Things/Industrial Wireless Network (IIoT/IWN) technologies are critical in smart factories and intelligent manufacturing systems [5][7][8]. They offer advantages like flexible network configuration, mobile device connectivity, and cost savings by eliminating the requirement for wires. As a result, the incorporation of wireless networks in an industry increases process productivity and efficiency.

However, interference, fading, security, and a restricted energy source limit their usefulness. Because of metallic surfaces and barriers, industrial environments present extra challenges to wave propagation [9]. While wireless technology have gained acceptance in some industrial applications, more study is required for essential applications. To meet Industry 5.0 criteria, low-power wireless technologies such as ZigBee, Xbee, and Bluetooth are available [10]. Long-range wireless control and monitoring capabilities (at indoor/urban ranges of up to 60 m and outdoor sight ranges of up to 1200 m) are provided by Xbee modules operating in the 2.4-2.5 GHz frequency range.

This study aims to design and develop a wireless acquisition system (SensorNet) for distributed sensor measurement in industrial applications. The designed and developed laboratory experimental test setup is shown in Fig. 1. The system customizes sensor array size, data resolution, and synchronization. In this proposed system, all

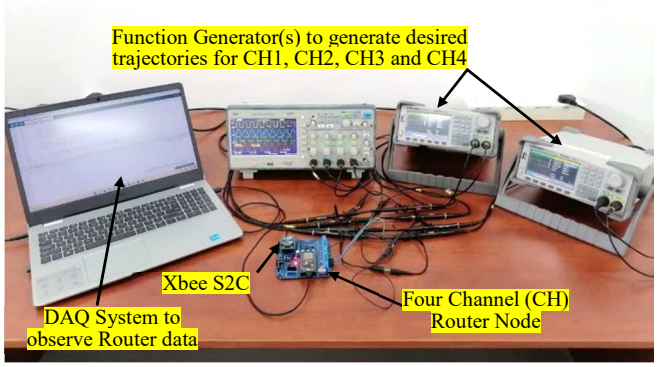

Fig. 1. Experimental test setup of the SensorNet

sensors are connected to a router node, and the transmitted data is received via a coordinator node. The number of sensors connected to a router node and the number of router nodes in the network can be customized. Simultaneously, the resolution of the transmitted data must be considered when adjusting the size of the sensor array [11]. To validate and evaluate the performance of the developed system, experimental measurements were conducted.

The paper is organized as follows: Section II presents the overall design, while Section III details the system architecture. Sections IV, V, and VI describe the systematic approach for decoding the observed data, test results, and conclusion, respectively.

II. Overall System Design

The proposed SensorNet system is intended to be scalable and configurable in order to fulfill the individual requirements of the many applications depicted in Fig. 2. It permits simple adjustments to the number of sensors, data resolution, data synchronization, and router nodes within the network. The system design comprises of a series of sensors connected to router nodes, which provide data to a central coordinator node via the IEEE 802.15.4 protocol [2].

The transmitted data is decoded and examined using the proposed mechanism. This study describes the wireless acquisition of data using various numbers of analog/digital sensors, such as one, three, and four, with an 8-bit data resolution. Researchers can construct customized data monitoring systems more quickly by investigating the proposed mechanism.

III. Implementation of the Sensor Network

The Xbee devices have two modes of communication:

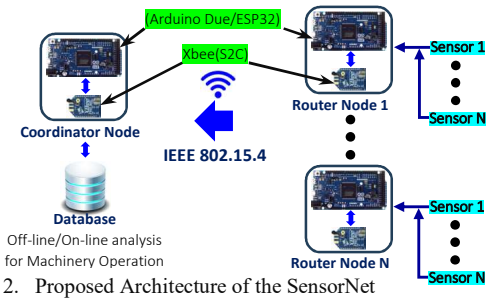

Fig. 2. Proposed Architecture of the SensorNet

Application Transparent (AT) mode and the Application Programming Interface (API) mode [12]. The proposed SensorNet, shown in Fig. 1, is based on the API mode, where data is communicated in packets (API frames). This mode is suitable for industrial applications [13] as it allows the formation of larger networks for tasks like data collection from multiple locations and industrial machinery automation. Before establishing wireless connectivity between the Coordinator(s) and the Router(s), as discussed in Section II, the Xbee modules require configuration.

A. Configuration of Xbee as Coordinator and Router

The configuration settings for the Xbee modules designated as the Coordinator and Router, when used in API mode, are shown in Table 1. Both nodes have the Personal Area Network ID (PAN ID) set to '123' (any value within the 16-bit range) and the Channel (CH) set to 'C' (Channel setting). According to Table 1, "Coordinator" and "Router" were assigned as the coordinator and router's Node Identifiers (NI), respectively. These nodes can send and receive data through the serial interface via the API mode setting. In addition, these nodes communicate information such as the source/destination of the data or signal quality in addition to transmitting and receiving sensor data.

B. API Data Frame Structure

The API data frame structure is shown in hexadecimal and decimal notations, respectively, in Fig. 3. The four blocks that make up the API data frame specify its type and the way the data is organized in the field [8].

The first block which is the “Start delimiter” that represent the values “$7E_{16}$” and “126_{10}” in hexadecimal and decimal in the API frame is the starting byte of the bit sequence. The second byte and the third byte in block 2 contained information about the length of the received data via the Coordinator node. The third block represents the API-data

TABLE I. CONFIGURATION OF PARAMETERS OF THE COORDINATOR AND ROUTER NODES

Parameter Name	Coordinator	Router
Chanel (CH)	C	C
PAN ID (ID)	123	123
Coordinator Enable	Coordinator [1]	End Device [0]
Node Identifier (NI)	Coordinator	Router
Interface Data Rate (BD)	230400 [7]	230400 [7]
API Enable (AP)	API enable [1]	API disabled [0]

structure, which is the core of the proposed protocol and contains multiple bytes based on the transmitted information via the Router node. The final block/fourth block is the checksum, which is a single byte that contains information about the transmission errors in proposed protocol.

IV. SYSTEMATIC APPROACH OF DECODING OBSERVED DATA

The proposed mechanism was implemented on an ESP32 DevKitC in the Arduino IDE, also used for IoT applications [14]. Prior to discussing the data decoding mechanism, the data transmitted format in API mode via the router node is explained. Fig. 4 and Fig. 5 illustrate the Arduino sketch (code/programme) developed for the Router enabling to transmit of observed data via one-sensor and, three-sensors and four-sensors within an 8-bit resolution respectively. Fig. 4 depicts the Arduino sketch responsible for retrieving an analog sensor value from ADC0 channel and subsequently transforming it into a range of 0 to 255, thereby acquiring the data within an 8-bit resolution. Finally, the execution of the "Serial.println(SensorValue, DEC)" command serves the purpose of storing the acquired data in the serial buffer, encapsulated as an integer value. This enables the Xbee S2C module to successfully transmit the data to the Coordinator node by means of an API data frame as American Standard Code for Information Interchange (ASCII) characters [15]. It becomes essential to distinguish the observed data in terms of the sensors from which they receive when many sensors are simultaneously transmitting data via the proposed protocol during the same sampling cycle (iteration). This condition is satisfied by the transmission of an ASCII character that has been pre-determined and assigned to each sensor individually (as the data address). Through this process, it is eventually able to pinpoint the precise source sensor associated with the sent observed data. Notably, for a three-sensor array, the ASCII letters 'A', 'B', and 'C' are carefully assigned to each individual sensor, whereas for a four-sensor array, the comparable assignment includes the characters ‘A’, ‘B’, ‘C’

7E (HEX)	0	A	81	0	0	14	0	31	33	35	D	A	BA
126 (DEC)	0	10	129	0	0	21	0	49	51	53	13	10	185
1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	9th Byte	10th Byte	11th Byte	12th Byte	13th Byte	14th Byte
Start delimiter	Length		API Frame type					Data (Hex bytes of ASCII text =>31|33|35 => 135_{10}) (Decimal bytes of ASCII text=>49|51|53=>135_{10})			Blank		Checksum
Block 1	Block 2		Block 3										Block 4

Fig. 3. Decoded API data observed via the Coordinator in hexadecimal/decimal.

```
int analogPin_1 = 36; // ADC0
int SensorValue_1 = 0;
void setup() {
  Serial.begin(230400);
  analogReadResolution(12);
}
void loop() {
SensorValue_1 = map(analogRead(analogPin_1), 0, 4095, 0, 255);
  delay(5);
  Serial.println(SensorValue_1, DEC);
  Serial.flush();
  delay(5);
}
```

Fig. 4. Arduino sketch for Router node for an one sensor in 8-bit resolution

and 'D'. Fig. 5 show the Arduino sketch corresponding to the three-sensor array and four-sensor array. When transmitting the observed data either via an analog or digital sensor(s) the proposed protocol is responsible for transmitting the data in 8-bit resolution. Therefore, for a same sampling iteration ‘k’ the minimum observed value is 0_{10} (00000000_2) and the maximum observed value is 255_{10} (11111111_2) from each sensor. That means the proposed algorithm in the Router node has mapped whatever the observed data into a value between 0 to 255 as shown in Fig. 4 and Fig. 5. Therefore, in this

proposed mechanism when the data is converted into decimal value(s) and when it transmits as ASCII characters the transmitted value will be a one-digit number (when the mapped data is between 0 to 9), two-digit number (when the mapped data is between 9 to 99) or a three-digit number (when the mapped data is between 99 to 255). This combination will take place for each sensor which is connected to the Router node. If these combinations are known as 'States' for a one-sensor scenario it has three states that means being transmitting a one-digit value, two-digit value and a three-digit value. For example, if two-sensor scenario considered for the same sample iteration 'k' the first sensor may transmit a one-digit decimal value while the second sensor may transmit a three-digit decimal value. Therefore, before decoding the transmitted data frame from the Coordinator node for each sampling iteration k, it is needed to identify all the possible number of states that could take place. When N is the maximum number of digits of the sensor value (here it is three due to 8-bit resolution and equal to 255 in decimal) and n is number of sensors attached to the Router node, then total number of sensor states (S_{ST}) for any combinations of sensors are possible to calculate for every sampling instant k as in (1).

$$S_{ST} = N^n \tag{1}$$

According to (1) if the maximum number of digits (N) are three of the observed sensor value (with 8-bit resolution) and the number of sensors (n) connected to the Router node is four then S_{ST} equal to $(3^4) = 81$. Table II shows the S_{ST} for number of four sensors (four sensor array) with 8-bit resolution (representing values between 0 to 255 in decimal).

```
int analogPins[] = {36,39, 34, 35};  // ADC0, ADC3, ADC6 & ADC7
int SensorValues[4] = {0};
void setup() {
  Serial.begin(230400);
  analogReadResolution(12);
}
void loop() {
   for(int i=0; i<4, i++){
   SensorValues = map(analogRead(analogPins[i]), 0,4095,0,255);
   delay(10);
   }

   for(int i=0; i<3, i++){ //$
   char sensorName = 'A' + i;
   Serial.println(sensorName);
   Serial.println(SensorValues[i], DEC)
   }
   delay(10);
  Serial.flush();
} //$ i<4 when four sensors are connected to the Router node.
```

Fig. 5. Arduino sketch for Router node for three sensors and four sensors in 8-bit resolution

TABLE II. TOTAL NUMBER OF SENSOR STATES (S_{ST}) FOR A FOUR SENSOR ARRAY

State No.	Sensor: A			Sensor: B			Sensor: C			Sensor: D		
	A3	*A2*	*A1*	*B3*	*B2*	*B1*	*C3*	*C2*	*C1*	*D3*	*D2*	*D1*
1			**1**			1			1			1
2		**1**				1			1			1
3	**1**					1			1			1
4			1		1				1			1
5		1			1				1			1
6	1				1				1			1
7			1	1					1			1
8		1		1					1			1
9	1			1					1			1
10			1			1		1				1
11		1				1		1				1
12	1					1		1				1
13			1		1			1				1
14		1			1			1				1
15	1				1			1				1
16			1	1				1				1
17		1		1				1				1
18	1			1				1				1
19			1			1	1					1
20		1				1	1					1
21	1					1	1					1
22			**1**		**1**		**1**					1
23		1			1		1					1
24	1				1		1					1
25			1	1			1					1
26		1		1			1					1
27	1			1			1					1
28			1			1			1		1	
29		1				1			1		1	
30	1					1			1		1	
31			1		1				1		1	
32		1			1				1		1	
33	1				1				1		1	
34			1	1					1		1	
35		1		1					1		1	
36	1			1					1		1	
37			1			1		1			1	
38		1				1		1			1	
39	1					1		1			1	
40			1		1			1			1	
41		1			1			1			1	
42	1				1			1			1	
43			1	1				1			1	
44		1		1				1			1	
45	1			1				1			1	
46			1			1	1				1	
47		1				1	1				1	
48	1					1	1				1	
49			1		1		1				1	
50		1			1		1				1	
51	1				1		1				1	
52			1	1			1				1	
53		1		1			1				1	
54	1			1			1				1	
55			1			1			1	1		
56		1				1			1	1		
57	1					1			1	1		
58			1		1				1	1		
59		1			1				1	1		
60	1				1				1	1		
61			1	1					1	1		
62		1		1					1	1		
63	1			1					1	1		
64			1			1		1		1		
65		1				1		1		1		
66	1					1		1		1		
67			1		1			1		1		
68		1			1			1		1		
69	1				1			1		1		
70			**1**	**1**				**1**		**1**		
71		1		1				1		1		
72	1			1				1		1		
73			1			1	1			1		
74		1				1	1			1		
75	1					1	1			1		
76			1		1		1			1		
77		1			1		1			1		
78	1				1		1			1		
79			1	1			1			1		
80		1		1			1			1		
81	1			1			1			1		

After identifying all the possible states as shown in Table II for a corresponding sensor array it needs to obtain the API data frames for each sensor state (for a four-sensor array it needs to obtain 81 API data frames). Fig. 6 depicts the Arduino sketch that was used to obtain the API data via the Coordinator node in decimal values according to the format shown in Fig. 3. To describe the data decoding mechanism of the proposed protocol, for example, three cases are discussed.

Therefore, Fig. 7, Fig. 8 and Fig. 9 shows the API data frames (in decimal values) obtained in the case of one-sensor, three- sensors and four-sensors array respectively. However,

describing about the decoding mechanism of API data frames for all 81 sensor states (S_{ST}) is a constrain due to page limitations. Therefore, to overcome this issue for all these three cases five sensor states of Table II (state 1, state 2, state 3, state 22 and state 70) have been taken into consideration.

A. Data Decodign of One Sensor with 8-bit Resolution

Fig. 7 depicts the API data structure obtained for state 1, state 2 and state 3 for the case of one sensor (data obtained via only Sensor A because the total number of states are three ($3^1 = 3$)) with one-digit number, two-digit number and three-digit number respectively. In Fig. 7 from 1st byte to 8th byte has been described in Section III. B. State 1, state 2 and state 3 observed data is represented by the 9th byte, 9th byte and 10th byte, and, 9th byte, 10th byte and 11th byte respectively as shown in Fig. 7. For all these three states ASCII characters '13' and '10' represent Carriage Return (CR) and Line Feed (LF) control characters. Using CR and LF appropriately in the proposed protocol ensures proper formatting and organization of data while transmitting from the Router node and receiving via the Coordinator node. Moreover, decimal values from 48 to 57 represent the ASCII values from 0 to 9 respectively. Therefore, a monitoring mechanism of CR and LF bytes enables us to distinguish one-digit number, two-digit number and three-digit number data. For the case of one sensor by continuously monitoring the 9th byte, 10th byte and 11th byte decimal values and assigned those values to an integer variables p, q and r respectively.

Therefore, when 'q == 13 && r == 10' (q equal 13 and r equal 10), 'q != 13 && r == 13' (q not equal 13 and r equal 13) and 'q != 13 && r != 13' (q not equal 13 and r, not equal 13) possible to correctly identify and distinguish a one-digit number, two-digit number and three-digit number respectively. In order to determine the observed data, the data decoding [16] system must first identify each decimal byte in the ASCII code. Fig. 10 illustrates the Arduino sketch that checks the value of variable 'x' against various decimal digits (48 to 57) representing ASCII codes for characters from '0' to '9'. If 'x' matches any of these digits, the corresponding value is assigned to the integer variable 'dec_y (where $1 \leq y \leq 3$). For example, if 'x' is equal to 48, 'dec_y' is assigned the value 0. Similarly, if 'x' is equal to 49, 'dec_y' is assigned the value 1, and so on. Therefore, for a one-digit number when '$x = p$' then '$y = 1$'. When $y = 1$ in the case of a one-digit number the observed data is assigned to the variable 'dec_1' as (2).

$$\text{Decemal}_{\text{val}} = (dec_1) \tag{2}$$

In the case of two-digit number when '$x = p$' and '$x = q$' then '$y = 1$' and '$y = 2$' respectively. Then the observed data is assigned to 'dec_1' and 'dec_2'. The position of each digit determines its value, with the rightmost digit 'dec_1' representing the ones place and the next digit 'dec_2' representing the tens place. Therefore, via the observed ASCII values for a two-digit number the observed data is calculated as (3).

$$\text{Decemal}_{\text{val}} = \big((dec_2 \times 10) + (dec_1)\big) \tag{3}$$

In the case of three-digit number when '$x = p$', '$x = q$' and '$x = r$' then '$y = 1$', '$y = 2$' and '$y = 3$' respectively. Then the observed data is assigned to 'dec_1', 'dec_2' and 'dec_3'. The position of each digit determines its value, with

```
void setup() {
     Serial.begin(230240);
}
void loop() {
if (Serial.available()>= 16){if (Serial.read() == 0x7E){
      Serial.print('1'); Serial.print('2'); Serial.print('6');
      Serial.print(",");
      for(int i=0;i<65;i++){Serial.print(Serial.read(),DEC);
      Serial.print(",");}
      Serial.println(); } } }
```

Fig. 6. Arduino sketch to decode the API data frame in decimal values

Sensor A	*State No. 1*	126	0	8	129	0	0	21	0	**56**			**13**	**10**	26
		1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	9th Byte			10th Byte	11th Byte	12th Byte
	State No. 2	126	0	9	129	0	0	22	0	**57**	**55**		**13**	**10**	255
		1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	9th Byte	10th Byte		11th Byte	12th Byte	13th Byte
	State No. 3	126	0	10	129	0	0	22	0	**50**	**50**	**54**	**13**	**10**	183
		1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	9th Byte	10th Byte	11th Byte	12th Byte	13th Byte	14th Byte
Frame description		Start delimiter	Length		API Frme type					Data (Decimal Byte)			Blank		Checksum

Fig. 7. API data frame for the case of Sensor-A (*one-sensor*) with 8-bit resolution

State No. 22	*Sensor A*	126	0	26	129	0	0	22	0	**65**	13	10	**52**		**13**	**10**	221
		1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	9th Byte	10th Byte	11th Byte	12th Byte		13th Byte	14th Byte	30th Byte
	Sensor B	126	0	9	129	0	0	22	0	**66**	13	10	**57**	**51**	**13**	**10**	221
		1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	15th Byte	16th Byte	17th Byte	18th Byte	19th Byte	20th Byte	21st Byte	30th Byte
	Sensor C	126	0	10	129	0	0	22	0	**67**	13	10	**49**	**56**	**50**	**13**	**10**	221
		1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	22nd Byte	23rd Byte	24th Byte	25th Byte	26th Byte	27th Byte	28th Byte	29th Byte	30th Byte
Frame description		Start delimiter	Length		API Frme type					Data (Decimal Byte)					Blank		Checksum

Fig. 8. API data frame for the case of Sensor-A, Sensor-B and Sensor-C (*three-sensors*) with 8-bit resolution (according to state Table II)

State No. 70	*Sensor A*	126	0	34	129	0	0	23	0	**65**	13	10	**55**		**13**	**10**	200
		1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	9th Byte	10th Byte	11th Byte	12th Byte		13th Byte	14th Byte	38th Byte
	Sensor B	126	0	34	129	0	0	23	0	**66**	13	10	**50**	**51**	**54**	**13**	**10**	200
		1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	15th Byte	16th Byte	17th Byte	18th Byte	19th Byte	20th Byte	21st Byte	22nd Byte	38th Byte
	Sensor C	126	0	34	129	0	0	23	0	**67**	13	10	**51**	**55**		**13**	**10**	200
		1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	23rd Byte	24th Byte	25th Byte	26th Byte	27th Byte		28th Byte	29th Byte	38th Byte
	Sensor D	126	0	34	129	0	0	23	0	**68**	13	10	**49**	**57**	**55**	**13**	**10**	200
		1st Byte	2nd Byte	3rd Byte	4th Byte	5th Byte	6th Byte	7th Byte	8th Byte	30th Byte	31st Byte	32nd Byte	33rd Byte	34th Byte	35th Byte	36th Byte	37th Byte	38th Byte
Frame description		Start delimiter	Length		API Frme type					Data (Decimal Byte)						Blank		Checksum

Fig. 9. API data frame for the case of Sensor-A, Sensor-B, Sensor-C and Sensor-D (*four-sensors*) with 8-bit resolution (according to state Table II)

```
if      (x==48){dec_y = 0;}
else if (x==49){dec_y = 1;}
else if (x==50){dec_y = 2;}
else if (x==51){dec_y = 3;}
else if (x==52){dec_y = 4;}
else if (x==53){dec_y = 5;}
else if (x==54){dec_y = 6;}
else if (x==55){dec_y = 7;}
else if (x==56){dec_y = 8;}
else if (x==57){dec_y = 9;}
else{dec_y = 0;}
```

Decoded data assigned integer variable.

Fig. 10. Arduino sketch that decodes the observed ASCII data into decimal.

the rightmost digit 'dec_1' representing the ones place, the next digit 'dec_2' representing the tens place and the final digit (leftmost digit) representing the hundred place. Therefore, via the observed ASCII values for a three-digit number the observed data is calculated as (4).

$$\text{Decemal}_{\text{val}} = \left((dec_3 \times 100) + (dec_2 \times 10) + (dec_1)\right) \quad (4)$$

Therefore, in the case of one-sensor state 1, state 2 and state 3 represent the decimal values 8, 97 and 256 respectively.

B. Data Decodign of Three Sensors with 8-bit Resolution

Fig. 8 depicts the API data structure obtained for state 22 for the case of three sensors where data was observed through Sensor A, Sensor B and Sensor C with one-digit number, two-digit number and three-digit number respectively. Similarly, in the case of one sensor, here the observed data could decode for a one-digit value via the 12th byte (for Sensor A), for a two-digit value via the 18th and 19th bytes (for Sensor B) and for a three-digit value via the 25th, 26th and 27th bytes (for Sensor C). However, before decoding the observed data it needs to identify the origin (address of the sensor) of the data to distinguish the data from each sensor. Therefore, for the case of state 22 by continuously monitoring the 9th, 13th, 14th, 15th, 20th, 21st, 22nd, 28th and, 29th bytes and assigned those bytes decimal values to an integer variables from s_1 to s_9 respectively.

Therefore, when satisfying the conditions '$s_1 == 65$ && $s_2 == 13$ && $s_3 == 10$' (s_1 equal 65, s_2 equal 13 and s_3 equal 10), '$s_4 == 66$ && $s_5 == 13$ && $s_6 == 10$' (s_4 equal 66, s_5 equal 13 and s_6 equal 10) and '$s_7 == 67$ && $s_8 == 13$ && $s_9 == 10$' (s_7 equal 66, s_8 equal 13 and s_9 equal 10) it is possible to correctly identify and distinguish a one-digit number, two-digit number and three-digit number respectively (where decimal values 65, 66 and 67 represents the ASCII characters 'A', 'B' and 'C' which are the assigned addresses of Sensor A, Sensor B and Sensor C respectively).

After checking these three conditions, the data decoding mechanism must identify each decimal byte in ASCII code to determine the observed data from each sensor as shown in Fig. 8. However, this procedure is same as the data decoding mechanism for a one-sensor when one-digit number, two-digit number and three-digit number is decoded as discussed in Section IV. A. Therefore, after decoding the data through this proposed mechanism for the case of three-sensor, state 22 represents the decimal values 4, 93 and 182 observed via Sensor A, Sensor B and Sensor C for the same sampling iteration k respectively.

C. Data Decodign of Four Sensors with 8-bit Resolution

Fig. 9 depicts the API data structure obtained for state 70 for the case of four sensors where data was observed through Sensor A, Sensor B, Sensor C and Sensor D with one-digit number, three-digit number and two-digit number and three-digit number respectively. Similarly, in the case of one sensor and three sensors, here the observed data could decode for a one-digit value via the 12th byte (for Sensor A), for a three-digit value via the 18th, 19th and 20th bytes (for Sensor B), for a two-digit value via the 26th and 27th bytes (for Sensor C) and for a three-digit value via the 33rd, 34th and 35th bytes (for Sensor D). Therefore, for the case of state 70 by continuously monitoring the 9th, 13th, 14th, 15th, 21st, 22nd, 23rd, 28th, 29th, 30th, 36th and 37th bytes and assigned those bytes decimal values to an integer variables from s_1 to s_{12} respectively.

Therefore, when satisfying the conditions '$s_1 == 65$ && $s_2 == 13$ && $s_3 == 10$' (s_1 equal 65, s_2 equal 13 and s_3 equal 10), '$s_4 == 66$ && $s_5 == 13$ && $s_6 == 10$' (s_4 equal 66, s_5 equal 13 and s_6 equal 10), '$s_7 == 67$ && $s_8 == 13$ && $s_9 == 10$' (s_7 equal 67, s_8 equal 13 and s_9 equal 10) and '$s_{10} == 68$ && $s_{11} == 13$ && $s_{12} == 10$' (s_{10} equal 68, s_{11} equal 13 and s_{12} equal 10) it is possible to correctly identify and distinguish a one-digit number, three-digit number, two-digit number and three-digit number respectively (where decimal values 65, 66, 67 and 68 represents the ASCII characters 'A', 'B', 'C' and 'D' the assigned addresses of Sensor A, Sensor B, Sensor C and Sensor D respectively). Similar to the three-sensor scenario, after identifying these four conditions, the data decoding mechanism must identify each decimal byte in ASCII code to determine the observed data from each sensor as shown in Fig. 9. This procedure is same as the data decoding mechanism for a one-sensor when one-digit number, two-digit number and three-digit number is decoded as discussed in Section IV. A. Therefore, after decoding the data through this proposed mechanism for the case of four-sensor, state 70 represents the decimal values 7, 236, 37 and 197 observed via Sensor A, Sensor B, Sensor C and Sensor D for the same sampling iteration k respectively.

V. Experimental Test Results

The response of the proposed SensorNet system and the algorithm, which were implemented on ESP32 DevKitC and Arduino IDE, was verified through a real-time experimental setup as shown in Fig. 1. Fig. 11 shows the designed and developed Coordinator Node and the Router Node utilized to obtain the test results. However, when considering the hardware agent of these two nodes, both of the developed boards are identical. Therefore, a simple hardware configuration process enables configuring the two boards as the Router node and the Coordinator node. During the experiment, a distance of 72 m was maintained in an outdoor [17] environment. To analyze the performance of the proposed SensorNet system and the protocol in real-time, a sinusoidal wave was utilized via the developed protocol for a one-sensor scenario. Fig. 12 shows the responses were obtained via the Coordinator node and compared with the reference trajectories that were fed to the Router node. Fig. 13 shows the observed trajectory response when the desired trajectory [18][19] frequency is greater than 20Hz. Fig. 14 shows the decoded data for the all the four-sensors.

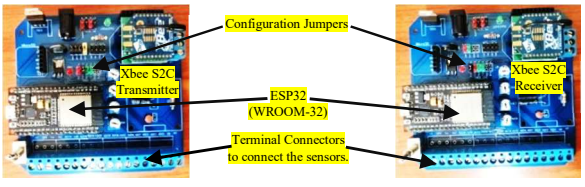

Fig. 11. Router and Coordinator Nodes which is responsible for transmit and decode the observed data respectively

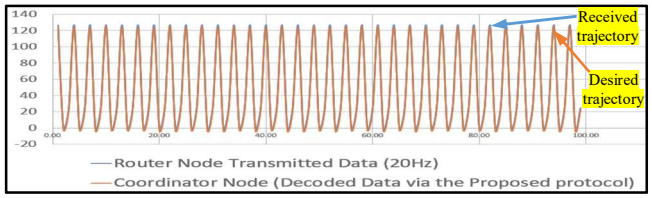

Fig. 12. Desired trajectory (sinusoidal wave) transmitted via the Router node and the decoded trajectory via the Coordinator node

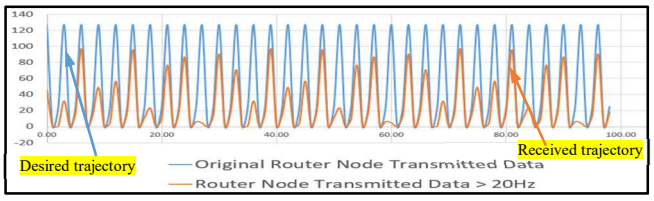

Fig. 13. Trajectory responses when the frequency is greater than 20Hz

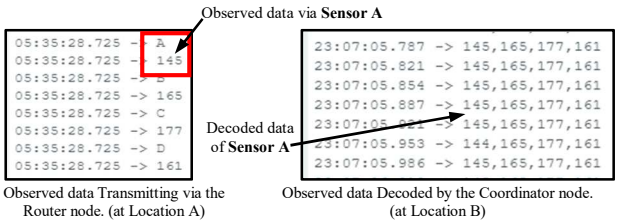

Fig. 14. The transmitted and the received data via the proposed SensorNet mechanism for the all four-sensors.

VI. Conclusion

In this research, a wireless sensing algorithm for Xbee S2C modules via ESP32 in Arduino IDE for Industrial applications has been developed and tested. Moreover, the proposed mechanism offers customizable data resolution, sensor array size and synchronization at the same sampling iteration. The proposed algorithm was designed and developed to decode data up to four-sensors with 8-bit resolution but not limited to this configuration. If more sensors or a higher resolution are needed, it can be expanded. Additionally, it demonstrates that the proposed approach was successful for a sinusoidal trajectory with a frequency maintained up to 20 Hz.

Furthermore, additional research should aim to streamline the program algorithm by creating and refining a generic or universal algorithm capable of accommodating various numbers of states, adjustable sensor quantities, and adaptable sensor resolutions.

References

[1] P. Fraga-Lamas, J. Varela-Barbeito, and T. M. Fernandez-Carames, "Next Generation Auto-Identification and Traceability Technologies for Industry 5.0: A Methodology and Practical Use Case for the Shipbuilding Industry," *IEEE Access*, vol. 9, pp. 140700–140730, 2021, doi: 10.1109/ACCESS.2021.3119775.

[2] H. R. Jayetileke, W. R. de Mel, and H. U. W. Ratnayake, "Dynamic Optimization Self-adaptive AI Controller for a Four-wheel Independent Drive Electric Rover," *AIT*, vol. 1, no. 1, May 2021, doi: 10.31357/ait.v1i1.4873.

[3] H. R. Jayetileke, W. R. De Mel, and S. C. Mukhopadhyay, "Real-Time Metaheuristic Algorithm for Dynamic Fuzzification, De-Fuzzification and Fuzzy Reasoning Processes," *Applied Sciences*, vol. 12, no. 16, p. 8242, Aug. 2022, doi: 10.3390/app12168242.

[4] H. R. Jayetileke, W. R. de Mel, and H. U. W. Ratnayake, "A Dynamic AI Controller for a Field-Oriented Controlled BLDC Motor to Achieve the Desired Angular Velocity and Torque," *International Journal of Intelligent Systems and Applications in Engineering*.

[5] S. B. Takale and S. D. Lokhande, "Quality of Service Requirement in Wireless Sensor Networks: A Survey," in *2018 IEEE Global Conference on Wireless Computing and Networking (GCWCN)*, Lonavala, India: IEEE, Nov. 2018, pp. 34–38. doi: 10.1109/GCWCN.2018.8668636.

[6] K. F. Haque, A. Abdelgawad, and K. Yelamarthi, "Comprehensive Performance Analysis of Zigbee Communication: An Experimental Approach with XBee S2C Module," *Sensors*, vol. 22, no. 9, p. 3245, Apr. 2022, doi: 10.3390/s22093245.

[7] S. Pirbhulal *et al.*, "A Novel Secure IoT-Based Smart Home Automation System Using a Wireless Sensor Network," *Sensors*, vol. 17, no. 12, p. 69, Dec. 2016, doi: 10.3390/s17010069.

[8] D. J. Ahn and J. Jeong, "A Bigdata Search Engine Based Cloud Computing Network Architecture in Smart Factory Environment," in *2019 2nd International Conference on Intelligent Autonomous Systems (ICoIAS)*, Singapore, Singapore: IEEE, Feb. 2019, pp. 91–95. doi: 10.1109/ICoIAS.2019.00023.

[9] D. Meier *et al.*, "Millimeter-Wave Radar Sensor for Automated Tomographic Imaging of Composite Materials in a Manufacturing Environment," *IEEE Sens. Lett.*, vol. 5, no. 3, pp. 1–4, Mar. 2021, doi: 10.1109/LSENS.2021.3055060.

[10] Y. Zhou, X. Liu, C. Liu, H. Guo, and X. Yang, "Low-Power Polling Strategy for Wireless Sensor Nodes Based on Quasi-Synchronization Technology," in *2019 IEEE MTT-S International Wireless Symposium (IWS)*, Guangzhou, China: IEEE, May 2019, pp. 1–3. doi: 10.1109/IEEE-IWS.2019.8804131.

[11] S. Davila-Montero, S. Parsnejad, Y. Gtat, and A. J. Mason, "Improving Object Localization Resolution using a Wearable Ultrasonic Sensor Array," in *2020 IEEE 63rd International Midwest Symposium on Circuits and Systems (MWSCAS)*, Springfield, MA, USA: IEEE, Aug. 2020, pp. 237–240. doi: 10.1109/MWSCAS48704.2020.9184439.

[12] "What is API (Application Programming Interface) Mode and how does it work? | Digi International." Accessed: Jun. 30, 2023. [Online]. Available: https://www.digi.com/support/knowledge-base/what-is-api-application-programming-interface-mode

[13] S. Secil *et al.*, "Design of an application programming interface (API) for commercial industrial robots," in *2016 IEEE 14th International Conference on Industrial Informatics (INDIN)*, Poitiers, France: IEEE, Jul. 2016, pp. 225–230. doi: 10.1109/INDIN.2016.7819163.

[14] R. P. D. T. Rathnayaka, K. V. J. P. Ekanayake, H. U. W. Rathnayake, and H. R. Jayetileke, "Fleet management with real-time data analytics," in *2021 6th International Conference on Information Technology Research (ICITR)*, Moratuwa, Sri Lanka: IEEE, Dec. 2021, pp. 1–6. doi: 10.1109/ICITR54349.2021.9657406.

[15] C. Meng and L. Zhou, "Big Data Encryption Technology Based on ASCII And Application On Credit Supervision," in *2020 International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE)*, Fuzhou, China: IEEE, Jun. 2020, pp. 79–82. doi: 10.1109/ICBAIE49996.2020.00023.

[16] M. A. M. Albreem and M. F. M. Salleh, "Lattice sphere decoding technique for block data transmission systems," in *Proceeding of the 2011 IEEE International Conference on Space Science and Communication (IconSpace)*, Penang, Malaysia: IEEE, Jul. 2011, pp. 302–304. doi: 10.1109/IConSpace.2011.6015905.

[17] M. A. Fadzilla, A. Harun, and A. B. Shahriman, "Localization Assessment for Asset Tracking Deployment by Comparing an Indoor Localization System with a Possible Outdoor Localization System," in *2018 International Conference on Computational Approach in Smart Systems Design and Applications (ICASSDA)*, Kuching: IEEE, Aug. 2018, pp. 1–6. doi: 10.1109/ICASSDA.2018.8477602.

[18] G. Zhiwei, D. Jianfeng, and D. Feng, "Simulation research on trajectory tracking controller based on MPC algorithm," in *2017 2nd International Conference on Robotics and Automation Engineering (ICRAE)*, Shanghai: IEEE, Dec. 2017, pp. 212–216. doi: 10.1109/ICRAE.2017.8291382.

[19] D. Qiu, D. Qiu, B. Wu, M. Gu, and M. Zhu, "Hierarchical Control of Trajectory Planning and Trajectory Tracking for Autonomous Parallel Parking," *IEEE Access*, vol. 9, pp. 94845–94861, 2021, doi: 10.1109/ACCESS.2021.3093930.